DELIVERY BACKLOG / RETROSPECTIVE + NEXT RELEASE

From product intent to acceptance

Photo Viewer | Epics, features, user stories and acceptance criteria

Product owner: **Nikhil Sharma**
Prepared with assistance from Codex | Snapshot: 11 October 2026

Epic	Outcome / feature groups	PRD
E01	Open and reuse: Finder/drop entry, window lifecycle	R1-R2
E02	Browse: ordering, mixed formats, folder recovery	R3-R4
E03	Inspect: Fit, manual zoom, pixel scale and panning	R5
E04	Understand: file identity, metadata, compact zoom UI	R6
E05	Trust: read-only originals, decoding, recoverable errors	R7
E06	Stay responsive: async work, preview decode, queue, cache	R8
E07	Deliver: About, local packaging, maintainable quality	R9
E08	Publish: source hygiene, license, reproducibility and security	R10

How to read the backlog

Current describes behaviour implemented in the present source; it does not imply every criterion was manually re-tested for this document. **Proposed** is work still to do. P0 is the core trust/viewing contract or a publication gate; P1 is refinement or hardening. These priorities are retrospective, not historic sprint records.

Each story names a feature, user benefit, numbered acceptance criteria and a verification route. Automated evidence refers to the existing tests; manual checks and new tests are explicitly distinguished. No story points, delivery dates or invented team velocity are assigned.

E01 / R1-R2

Open once. Reuse the window.

Success: the user can enter from Finder or a drop without spawning a separate viewer for every photo.

S01 | F01 Entry: Finder open requests

P0 / Current

As a Mac user, I want Finder to open images in Photo Viewer so I can keep my existing file workflow.

1. Given one or multiple supported file URLs, when macOS sends an open event, route them into the shared application state.
2. A new external request replaces the active image/folder context and resets zoom to Fit.
3. Repeated requests reuse the same image window; files from different folders do not create additional image windows.

Verification: ApplicationState external-open tests; manual Finder single/multiple selection and repeated-open checks.

S02 | F02 Empty state: Drop-to-open

P0 / Current

As a user launching without a file, I want a quiet starting state and drag opening so I do not need an extra button.

1. With no selected image, show "Open an image from Finder" in the image area.
2. Dropping supported local file URLs opens an image in the existing viewer using external-open semantics.
3. Do not add File Open, library import or other browsing buttons; unsupported input must not crash the app.

Verification: Manual empty-state, drag/drop and unsupported-input smoke checks; state tests cover unsupported open.

S03 | F03 Window lifecycle: Large, normal macOS window

P0 / Current

As a MacBook user, I want a large resizable window with familiar controls so viewing does not take over my desktop.

1. Without a saved frame, occupy about 90% of the usable display; enforce the 640 x 480 point minimum.
2. Restore the saved window frame on subsequent launches and retain close, minimise and zoom controls.
3. Green-button zoom must resize normally, not enter macOS fullscreen. Reopen restores the same image window.
4. Closing the last window does not quit the app; Cmd-Q does. The optional About panel is not another image viewer.

Verification: Source contract plus manual fresh-preferences, relaunch, resize, green-button, minimise and reopen checks.

E02 / R3-R4

Browse the folder predictably

Success: filenames and supported formats, not accidental key focus or an extension mismatch, determine navigation.

S04 | F04 Folder sequence: Natural ordering

P0 / Current

As a viewer, I want left/right navigation to follow a predictable folder sequence so I can browse without switching to Finder.

1. Include supported image candidates in the immediate parent folder, ordered with natural filename comparison; photo2 precedes photo10.
2. Right selects the next item; Left selects the previous. At either endpoint remain on the current item without wrapping.
3. Preserve Fit or the selected manual percentage during navigation; update Photo x of y from the same ordered snapshot.

Verification: ApplicationState mixed-format/no-wrap and cached-position tests; ImageCatalog natural-order tests.

S05 | F05 Input and formats: Mixed files, same keys

P0 / Current

As a user with mixed photos, I want arrows to work across common formats so I need not open each type separately.

1. Recognise common JPEG/JPG, PNG, BMP, HEIC/HEIF, TIFF, GIF and native WebP candidates, including mixed-case extensions.
2. Recognise supported additional native formats; omit RAW and non-image files. Do not count directories disguised with image suffixes.
3. Unmodified arrow presses and repeats navigate; modified arrows and any active text field editor retain their standard behaviour.

Verification: Catalog format/type/symlink tests, loader tests and NavigationWindow key tests. Decoder advertisement is not exhaustive variant validation.

S06 | F06 Folder recovery: Permission and change handling

P0 / Current

As a user, I want folder-access failures and changes to be recoverable so the current photo never strands me.

1. If the parent cannot be read, keep the opened photo and show a navigation notice; retain selected same-folder files when available.
2. Retry access on navigation/refresh and clear the notice after success. A temporary refresh failure must not discard a usable cached list.
3. Refresh after relevant folder changes; boundary retry can discover a new sibling. Superseded folder results cannot replace the active context.

Verification: ApplicationState folder denial/retry/change tests and ImageCatalog folder-monitor tests; manual macOS permission check.

E03 / R5

Inspect at an understandable scale

Success: automatic Fit and deliberate manual zoom are distinct, predictable modes.

S07 | F07 Fit: Automatic sizing

P0 / Current

As a viewer, I want the complete photo fitted to the viewport so it is immediately usable.

1. Every external open selects Fit; resizing recomputes scale from original oriented pixel dimensions and backing scale.
2. Show actual Fit percentage even below 20% or above 200%; manual slider bounds do not constrain Fit.
3. Filled, checked Fit means active; manual changes deactivate it; clicking Fit restores automatic sizing.

Verification: Geometry/scale tests and manual active/inactive Fit visual check.

S08 | F08 Manual zoom: 5% controls

P0 / Current

As a viewer, I want slider and +/- controls in small steps so I can adjust scale without typing.

1. Slider values snap to the nearest 5% and clamp to 20-200%; there is no percentage text-entry field.
2. In manual mode each +/- action moves one 5% step, with no movement beyond an endpoint.
3. From out-of-range Fit, stepping enters the valid manual range; controls disable when no change is possible.

Verification: ZoomPolicy rounding/clamp/step tests and ApplicationState zoom control tests.

S09 | F09 Pixel geometry: True 100%

P0 / Current

As a detail reviewer, I want 100% to mean one image pixel per screen pixel so sharpness has a consistent meaning.

1. Use original image dimensions, not the downsampled preview, for manual and Fit geometry.
2. Recalculate point geometry when moving between Retina and non-Retina backing scales.
3. A higher-resolution replacement of the same photo preserves the panned centre and zoom state.

Verification: NavigationWindow geometry at both scales and same-photo upgrade tests; physical mixed-screen check remains manual.

S10 | F10 Panning: Reach enlarged detail

P0 / Current

As a viewer, I want mouse and trackpad panning so I can inspect areas outside the viewport.

1. When the manual image exceeds the viewport, mouse dragging and trackpad scrolling expose off-screen areas.
2. Centre the image when smaller than the viewport; avoid unintended navigation while panning.
3. Navigating to a different image does not reuse an unrelated panned offset.

Verification: ImageScrollView source contract; manual mouse, trackpad, resize and navigation smoke checks.

E04 / R6

Make identity and state legible

Success: the compact header answers which file, where it lives, its context and the current scale.

S11 | F11 File identity: One name/path line

P1 / Current

As a viewer, I want filename and path together so I can identify the image without losing space to extra rows.

1. Show filename first, then a dot separator and full path on one header row.
2. Text is noneditable. Long names/paths truncate gracefully while help and accessibility text expose the complete identity.
3. Replace both values when the active image changes; no visible app version number is added.

Verification: ContentView inspection plus narrow/wide-window and long-path visual checks.

S12 | F12 Metadata: One context line

P1 / Current

As a viewer, I want creation date, size and position together so I can understand the file without opening another tool.

1. Show Created date, Size and Photo x of y in that order, separated on one line below identity.
2. Read filesystem creation date and logical byte size; preserve a valid zero-byte size. Do not label this EXIF capture date.
3. Update metadata asynchronously; stale results cannot replace the current image's values. Show loading/unavailable states rather than invented values.
4. Position reflects supported candidates in the current accessible list, not all files on disk; a candidate may subsequently fail to decode.

Verification: ApplicationState metadata update, stale-result, unavailable, zero-byte and logical-size tests; layout check.

S13 | F13 Zoom legibility: Label follows the thumb

P1 / Current

As a viewer, I want scale near the slider thumb so I do not mistake it for the + button's value.

1. Place a small, single-line percentage above the thumb; remove the old far-right Fit/percentage text.
2. Align percentage, Min 20% and Max 200% captions vertically; give - and + equal icon/control geometry.
3. Use stable numeric typography; labels must not wrap when changing from two to three digits.
4. In Fit show actual percentage while thumb position snaps/clamps to the manual range; endpoint padding keeps labels within the control.

Verification: ZoomSliderMetrics geometry tests plus visual checks at endpoints, 20/95/100/200% and out-of-range Fit.

E05 / R7

Preserve trust and recover safely

Success: viewing never becomes an editing workflow, and a bad image does not stop folder browsing.

S14 | F14 Read-only contract: Preserve the original

P0 / Current

As the file owner, I want viewing to leave my images unchanged so I can inspect them with confidence.

1. Expose no save, rename, delete, rotate, annotation or metadata-edit action in the viewer or menus.
2. Application image access uses read-only operations; comparing controlled fixtures before/after yields the same contents, names, size, creation/modification dates and xattrs.
3. Explain that non-sandboxed read-only application behaviour is not an OS-enforced read-only permission boundary.

Verification: Source review; EXIF non-write test; synthetic benchmark invariance. Repeat representative fixture comparisons on release.

S15 | F15 Rendering semantics: Orientation, colour and first frame

P0 / Current

As a viewer, I want photos to look correct without playback controls or transformations saved to disk.

1. Apply embedded orientation to the displayed image and original-dimension geometry without rewriting the source.
2. Use the native colour-aware image path; validate tagged-colour fixtures as part of visual release QA.
3. Animated and sequence files display the first frame only; do not add playback controls.

Verification: ImageLoader EXIF and animated GIF tests; common-format tests. Colour-profile comparisons and all sequence variants need manual fixtures.

S16 | F16 Failure recovery: Bad or removed image

P0 / Current

As a viewer, I want a minimal error and working navigation so one broken file cannot block the rest of the folder.

1. A decode failure or missing file shows a readable error in the image area instead of terminating the viewer.
2. Keep navigation available to other candidates; an unreadable file does not silently destroy the folder context.
3. If manual-resolution upgrade fails, show the failure and retain browsing capability. Stale load results must never replace a newer selection.

Verification: ApplicationState corrupt-image and missing-upgrade tests, plus pipeline supersession tests. Oversized-input hardening is proposed in E08.

E06 / R8

Keep the interface responsive

Success: repeated browsing avoids repeated expensive work, and speculative work stays bounded.

S17 | F17 Folder index: Cache navigation work

P1 / Current

As a viewer in a large folder, I want arrows to reuse the index so the UI does not rescan for every image.

1. Enumerate, natural-sort and build a canonical index off the main thread; begin selected-image loading without waiting for discovery.
2. Repeated navigation/position lookup uses the cached snapshot; deduplicate aliases while retaining useful alias indices.
3. Folder notification, debounce and foreground/boundary refresh update state without applying obsolete results.

Verification: Cached-no-rescan, slow-scan/supersession, alias and monitor tests.

S18 | F18 Fit decode: Load only needed preview detail

P1 / Current

As a viewer, I want large photos to open efficiently while retaining full detail when I zoom.

1. Fit requests a display-sized decode using the viewport/backing scale; never upscale a small image during preview decoding.
2. Keep original oriented pixel dimensions independent of the preview. Manual zoom requests original detail.
3. While upgrading the same image, keep its preview visible and preserve scale/panned centre when replacement is ready.

Verification: ImageLoader downsample tests, ApplicationState upgrade test and NavigationWindow centre test.

S19 | F19 Decode scheduling: Bound obsolete work

P1 / Current

As a rapid browser, I want the newest requested image prioritised so work does not queue indefinitely.

1. Allow only one active decode and one latest pending request; newer requests replace obsolete pending work.
2. Do not display superseded results. An active synchronous ImageIO operation may finish; do not claim it is forcibly cancelled.
3. Do not speculatively decode/download neighbouring files.

Verification: ImageLoaderPipeline superseded-queue/single-decoder test and source review.

S20 | F20 Revisit cache: Bound photo caching

P1 / Current

As a user, I want fast revisits without an ever-growing photo cache on my drive.

1. Keep decoded revisit entries only in RAM, with a 128 MiB decoded-byte LRU budget; evict older entries and skip oversized cache entries.
2. Invalidate cached entries when file identity, size or modification metadata changes; quit releases the process cache.
3. Document that the cache is not total RAM: displayed images, pending results and decode allocations are additional.

Verification: ImageLoader cache reuse/invalidation, eviction and oversized-entry tests; inspect for persistent cache writes.

E07 / R9

Deliver a clear, maintainable utility

Success: a local build is easy to identify, use and rebuild, without expanding the main viewer controls.

S21 | F21 About: Credit human and AI work

P1 / Current

As the creator, I want an About section that credits ownership and assistance transparently.

1. Photo Viewer > About Photo Viewer opens one reusable, closable, nonresizable panel; add no About button to the image header.
2. Show "Created by Nikhil Sharma" and "Created with assistance from Codex, OpenAI's AI coding assistant."
3. Show Last updated using the build-recorded date; future builds update it, merely opening the app on a later day does not.
4. Show no visible version number; missing/invalid date metadata yields Unavailable rather than an invented date.

Verification: NavigationWindow About wording/date/panel tests and previous visual About inspection.

S22 | F22 Local package: Build and install reliably

P1 / Current

As a Mac owner, I want a Release app and source project so I can install and rebuild my own viewer.

1. Deliver the buildable Xcode project, shared test scheme and Release app for arm64 with macOS 26 deployment target.
2. Use optimised Release code with coverage instrumentation disabled; document the actual tested toolchain separately from the target OS.
3. Use an unversioned visible app name, Photo Viewer. Keep one intended installed copy to avoid duplicate Finder Open With registrations.
4. Disclose personal/ad-hoc signing status; do not describe this artifact as a notarised public release.

Verification: Project/settings, Release binary/signing and package checks; manual Finder registration check.

S23 | F23 Quality baseline: Keep changes reviewable

P1 / Current

As the product owner, I want tests tied to the product contract so refinements do not silently break core behaviour.

1. Keep tests for state, catalog, loader, keyboard/window geometry and zoom policy in the shared scheme.
2. Retain benchmark methodology and before/after claims with fixture, toolchain, timing and memory caveats.
3. Run the focused regression matrix before delivery; distinguish automated evidence, manual observations and unverified claims.

Verification: Latest saved run: 53 tests, 0 failures. Performance JSON and the release matrix on page 10.

E08 / R10 / PROPOSED

Prepare an open-source release

Success: publication contains intentional source and honest claims, with an explicit reuse and support policy.

S24 | F24 Source hygiene: Publish only intended files

P0 / Proposed

As the owner, I want clean source so private data and generated artifacts never become public.

1. Stage an allowlist: app source/assets, tests, Xcode project/shared scheme, README and approved documentation. Review every staged file.
2. Exclude work/DerivedData, logs, xcuserdata, app bundles, archives, personal images, credentials, profiling output, __MACOSX metadata and private paths.
3. Add .gitignore and a local secret scan; review new Git history and author email. Confirm artwork/source provenance before licensing.

Verification: Pending: file inventory, secret scan, provenance review and owner approval.

S25 | F25 Governance: License and explain the project

P0 / Proposed

As a prospective user or contributor, I want clear rights and build instructions so I can reuse the project responsibly.

1. Owner approves account, repository name, public visibility and license; add the chosen license before calling the release open source.
2. Correct permission/notarisation wording; document target OS, tested Xcode, supported-format limits, build/test steps and RAM-cache limits.
3. Add a private vulnerability-reporting route and contributor guidance; keep creator/AI credits and portfolio claims accurate.
4. Fresh-clone build/tests succeed without private files; OS-specific HEIC fixtures are documented or replaced with redistributable fixtures.

Verification: Pending: owner decisions, documentation review and clean-clone validation.

S26 | F26 Public hardening: Verify before broad distribution

P1 / Proposed

As a downloader, I want known limitations and basic hardening so hostile images and build assumptions do not surprise me.

1. Test pixel/decoded-byte limits or a bounded preview fallback; oversized/malformed inputs must not cause unbounded resource use.
2. Restrict decoding to local, readable regular files; define symlink handling and reject unsupported URL schemes or special files with tests.
3. Configure compatible macOS build/test CI and Swift CodeQL; check secret-scanning/push-protection settings and review alerts.
4. For a binary release, decide Developer ID signing, hardened runtime and notarisation; publish checksums and smoke-test clean installation.

Verification: Pending: new tests, CI/scanning run and binary-release decision. No claim of zero vulnerabilities.

VERIFICATION / RELEASE QUALITY

Definition of done

A story is done when its criteria are demonstrated by the appropriate test or observation, not just when code exists.

Coverage	Automated evidence now	Release check / remaining gap
Entry and window	External-open state behaviour; key handling; About reuse	Finder, drop, multi-select, closed/minimised reuse, saved geometry, no fullscreen
Folder browsing	Natural order, mixed types, permissions/retry, no wrap, snapshots	Desktop and protected-folder prompts; external changes and cloud-backed folders
Zoom and metadata	5% policy, geometry at both scales, stale metadata, slider coordinates	Mouse/trackpad pan, physical monitor change, long paths, caption alignment
Image semantics	Common-format fixtures, HEIC fixture, EXIF, GIF first frame	Colour profiles and additional format variants; HEIC fixture portability
Errors and resources	Corrupt/missing images, cache eviction, decode queue bounds	Malicious/oversized files, peak RAM, memory pressure and long-session behaviour
Read-only trust	Orientation test and synthetic file-invariance benchmark	Repeat representative contents/name/timestamp/xattr checks; fix misleading docs
Delivery	Saved run of 53 tests; prior local bundle/signature checks	Clean clone, license, asset provenance, CI, secret scan, public binary trust

Cross-cutting release gate

Build from a clean source checkout; run the shared test suite; record toolchain and skipped tests; complete relevant manual checks; review generated artifacts and source diff; verify originals remain unchanged; retain evidence. No debug logs, personal file paths or user images may appear in public examples.

Avoid false completion

A decoder advertising a file type does not prove every variant works. A memory-cache budget is not a total-memory guarantee. A passing unit suite is not a security audit. Native colour handling is not a completed colour-profile regression. The proposed E08 stories remain open.

Evidence map: PhotoViewerTests/ApplicationStateTests.swift (19), ImageCatalogTests.swift (8), ImageLoaderTests.swift (10), NavigationWindowTests.swift (10), ZoomPolicyTests.swift (6). Saved test run checked for this document: 11 October 2026, 53 tests, 0 failures; not rerun solely for PDF authoring.

PUBLICATION PLAYBOOK / PROPOSED

How to publish on GitHub

Recommended first release: reviewed source plus these product documents. A downloadable app is a separate distribution decision.

1. Agree the public identity and license

Choose the GitHub account and repository name (for example, photo-viewer-macos). Approve public visibility. MIT is a simple permissive option if broad reuse is intended; do not apply it without approval or before checking asset rights. Public visibility alone does not grant an open-source license.

2. Prepare a clean local repository

Create a source-only staging folder from the reviewed allowlist in S24, not the entire workspace. Add README, .gitignore, approved LICENSE, reporting/contribution guidance and a docs folder for the PDFs. Remove private paths from examples and use generated or licensed demo images.

3. Check before uploading

Run the test suite and a fresh-clone build with a compatible Xcode/macOS runner. Review all staged files and commit metadata, scan for credentials, correct permission claims and document limitations. Resolve findings or disclose them with a clear next action.

4. Publish with GitHub Desktop or Git

For a point-and-click flow, initialise the clean source folder as a local repository in GitHub Desktop, review Changes, make the initial commit and select Publish repository. Confirm the correct account/name, then choose public visibility only at the approved publication step. GitHub also documents equivalent Git/CLI commands.

5. Verify and maintain

Inspect the repository in the browser: expected files only, readable README, detected license and working build instructions. Check public secret-scanning alerts and account/repository push-protection settings. Add Swift CodeQL and macOS CI when a compatible runner is configured. Do not commit signing keys; manage any future signing secrets separately.

Publication status

Not published by this task. Required next decisions: GitHub account, repository name, license and explicit approval of the final staged contents. No passwords or tokens need to be pasted into chat.

Primary guidance / checked 11 October 2026

GitHub - Add locally hosted code
GitHub - License a repository | MIT overview
Secret scanning | Push protection | CodeQL (including Swift)